

PROJECT AND TEAM INFORMATION

Project Title

Virtual File System

Student / Team Information

<i>Team Name:</i> <i>Team #</i>	<i>Rootrix</i>
Team member 1 (Team Lead) <i>(Last Name, name: student ID: email, picture):</i>	<i>Naithani, Tanish –</i> <i>25100011477</i> <i>25100011477.geu.ac.in</i> 
Team member 2 <i>(Last Name, name: student ID: email, picture):</i>	<i>Singh, Tushar –</i> <i>2510011704</i> <i>2510011704@geu.ac.in</i> 
Team member 3 <i>(Last Name, name: student ID: email, picture):</i>	<i>Tayal, Rounak –</i> <i>2510370114</i> <i>2510370114@geu.ac.in</i> 
Team member 4 <i>(Last Name, name: student ID: email, picture):</i>	<i>Rawal, Aditya –</i> <i>2510380204</i> <i>2510380204@geu.ac.in</i> 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

Files and Folders are part of our daily computer use but the work behind simple file operations is usually hidden, when a file is created , changed , moved , removed the system must manage its location , size , permissions , directory positions and storage blocks. The operating system normally performs these tasks automatically.

this is why we selected a Virtual File System for the project. We want to build a small VFS in C++ that works inside our program and manages files directories and storage blocks without changing the computer actual file system . Implementing these components ourselves will help us understand how a file system works internally.

The project also gives us practical use of Data Structures and object oriented programming concepts studied in class. Directories can be represented through tree hashing can support faster lookup and a bitmap can track free and occupied blocks. Dividing the system into classes will keep different responsibilities organised.

We will also work with block allocation , file input and output , caching , LRU replacement , journaling , snapshots , and file system checking. The CLI will allow users to perform file and directory operations and observe the response of VFS.

State of the Art / Current solution (1 pt)

File management is normally handled by operating systems such as Windows and Linux. When a user creates, opens, edit, move, or delete a file, the operating system manages directories, storage space, file location, permissions, and metadata in background. Operating systems also provides mechanism for block management, caching, and recovery. These improve performance and help to reduce the effect of failures. However, the internal process is mostly hidden from users. We can manage files without seeing how their data is placed on storage blocks or how these blocks are allocated and released. Our project creates a small Virtual File System in C++ to make these concepts more easier to observe. It is not intended to replace an operating system's file system. Instead, it provides a controlled environment where we can implement basic file-system operations ourselves and connect it with the Data Structures, OOP, and Operating Systems concepts we studied in class.

Project Goals and Milestones (2 pts)

The main objective is to develop a functional Virtual File System (VFS) in C++ and understand the management of files, directories, storage blocks, and metadata using Data Structures and OOP. The system will provide:

1. A virtual disk divided into fixed-size blocks
2. A tree-based hierarchy for files and directories
3. Create, read, write, append, truncate, copy, move, rename, and delete operations
4. File information such as size, permissions, timestamps, and block locations
5. A bitmap for free and occupied blocks
6. Search using file names, paths, and metadata
7. An LRU cache for frequently accessed blocks
8. Journaling, file-system checking, and snapshots
9. A command-line interface for user interaction

Project Milestones

1. **System Design?** Define the architecture and required C++ classes and modules.

Virtual Disk and Block Management? Create fixed-size blocks and track allocation with a bitmap.

2. **File and Directory Management?** Build the hierarchy and implement file and directory operations.

3. **File Operations and Metadata?** Add read, write, append, and truncate while maintaining file information.

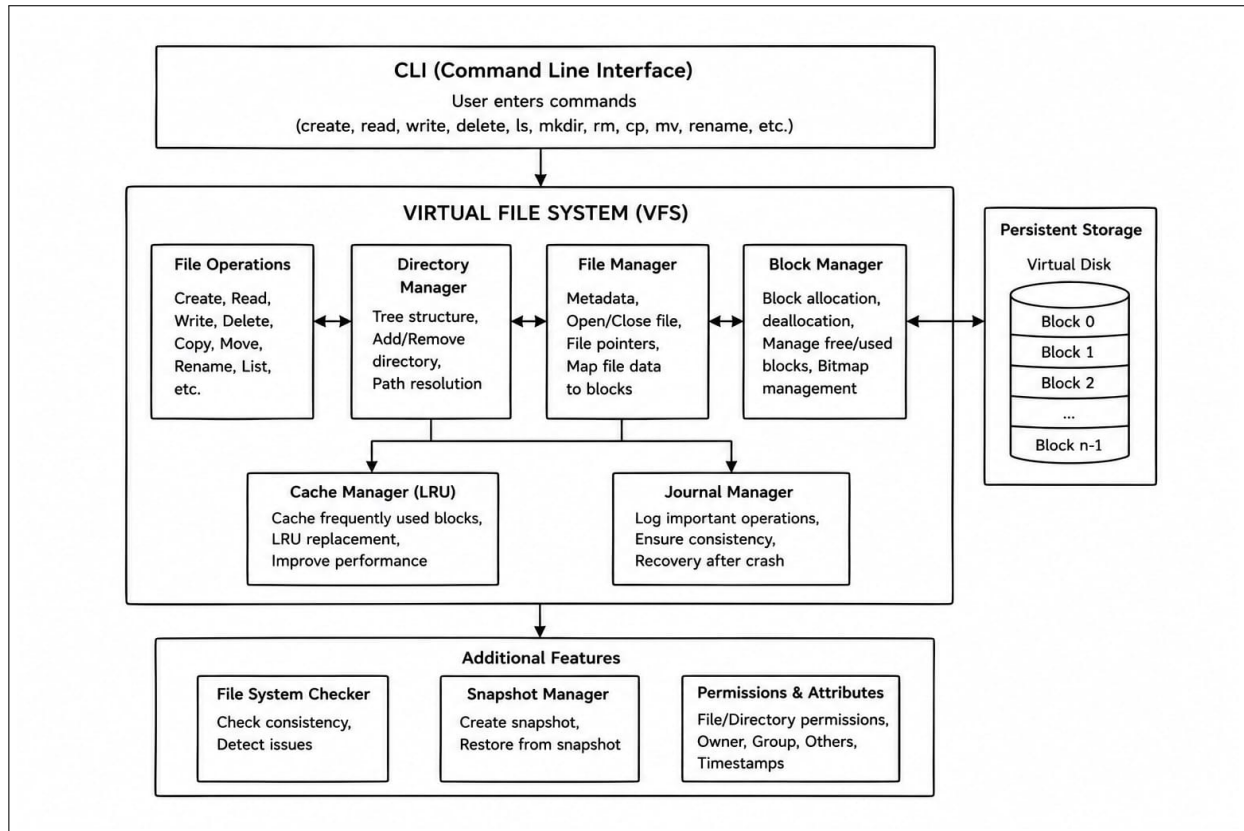
4. **Advanced Features?** Implement search, LRU caching, journaling, consistency checking, and snapshots.

5. **CLI, Testing and Integration?** Connect the modules and test the complete system through the CLI.

Project Approach (3 pts)

We plan to implement the Virtual File System in C++ using Object-Oriented Programming. The project will be divided into modules so each component can be developed and tested separately. The main components will be virtual disk, files, directories, block manager, cache, journal, and command-line interface. The virtual disk will contain equal-sized blocks. A bitmap will record whether blocks are free or occupied, while the block manager will allocate and release them when files change. Directories follow a tree structure, allowing them to contain files and subdirectories. Hash tables will support faster lookup where it is required. Each file object stores its name, size, type, permissions, timestamps, and data-block location. The CLI will support create, read, write, append, truncate, copy, move, rename, and delete operations. An LRU cache handles the frequently accessed blocks. Journaling will record operations, consistency checking will detect allocation problems, and snapshots will save and restore the file-system state. C++ file I/O will be used for virtual-disk storage and persistence.

System Architecture (High Level Diagram)(2 pts)



Project Outcome / Deliverables (1 pts)

The expected outcome is a working command line Virtual File System in C++. It will allow users to create and managing virtual files and directories without changing the computer actual file system.

1. The project is expected to provide:
2. A virtual disk with fixed-size blocks
3. A tree structure for files and directories
4. Create, read, write, append, truncate, copy, move, rename, and delete operations
5. File metadata and permission management
6. A bitmap for block allocation
7. Search for files and directories
8. An LRU cache for frequently accessed blocks
9. Journaling and basic recovery
10. File-system consistency checking
11. Snapshot creation and restoration
12. A command-line interface for system operations
13. Basic disk-usage and performance information

The project will apply trees, hash tables, bitmaps, linked structures, OOP, file I/O, memory management, caching, and basic file-system concepts in one practical system.

Assumptions

The virtual disk will have a fixed capacity and equal-sized blocks for storing file data. Files and directories will be managed inside the VFS. Each file will store its size, permissions, timestamps, and data-block locations. Users will perform operations through the C++ command-line interface.

The project is mainly intended for practical learning. The team will use directory trees, block allocation, metadata management, caching, journaling, and file-system checking to understand how these components work together.

References

- 1. Balagurusamy, E. Object Oriented Programming with C++. McGraw Hill Education. This book helped me learn about classes, objects, inheritance and other Object-Oriented Programming ideas that I used in the project.*
- 2. Kanetkar, Y. Data Structures Through C. BPB Publications. This book guided me on trees, linked lists, hashing and how to manage memory.*
- 3. GeeksforGeeks. Data Structure and Algorithms. This website helped me grasp data structures and how they can be used. Available at: <https://www.geeksforgeeks.org/data-structures/>*
- 4. Programiz. C++ Programming. This site served as a guide for C++ concepts, file handling and Object-Oriented Programming. Available, at: <https://www.programiz.com/cpp-programming>*